
AI AGENTS

WORKSHOP SERIES

Lora Yovcheva-Clavijo

UCLA MQE Spring 2026

SCAN THE QR CODE TO SIGN IN



Access to Materials: <https://github.com/lorayc/ai-agents>

THE WORKSHOP SERIES

01

Foundations

API calls, tool use, ReAct loop, FRED agent

02

RAG & Memory

Embeddings, vector stores, FOMC document search

03

Multi-Agent Systems

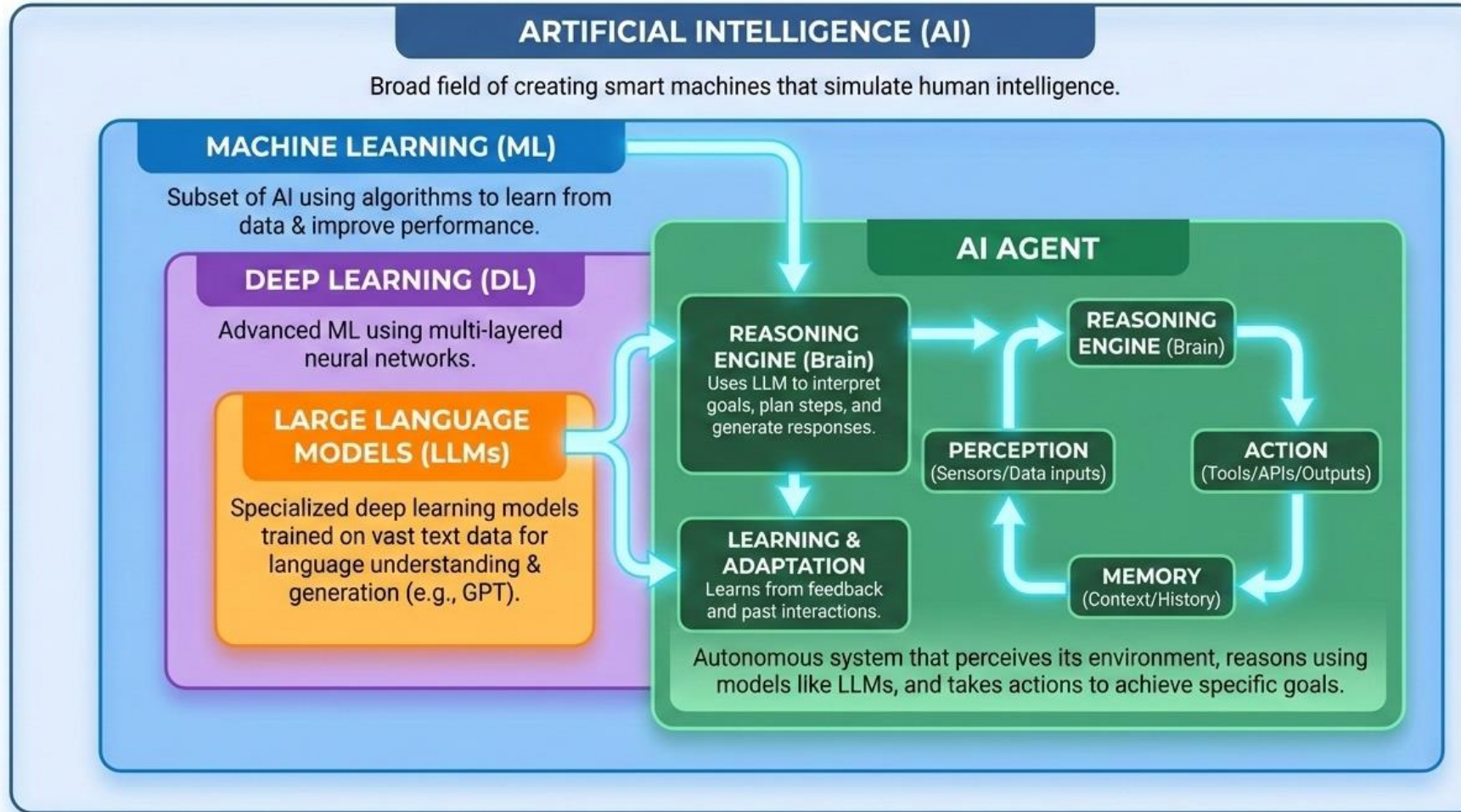
Orchestration, delegation, parallel research

04

Production & Eval

Guardrails, evaluation, deployment patterns

THE ECOSYSTEM



MOTIVATION



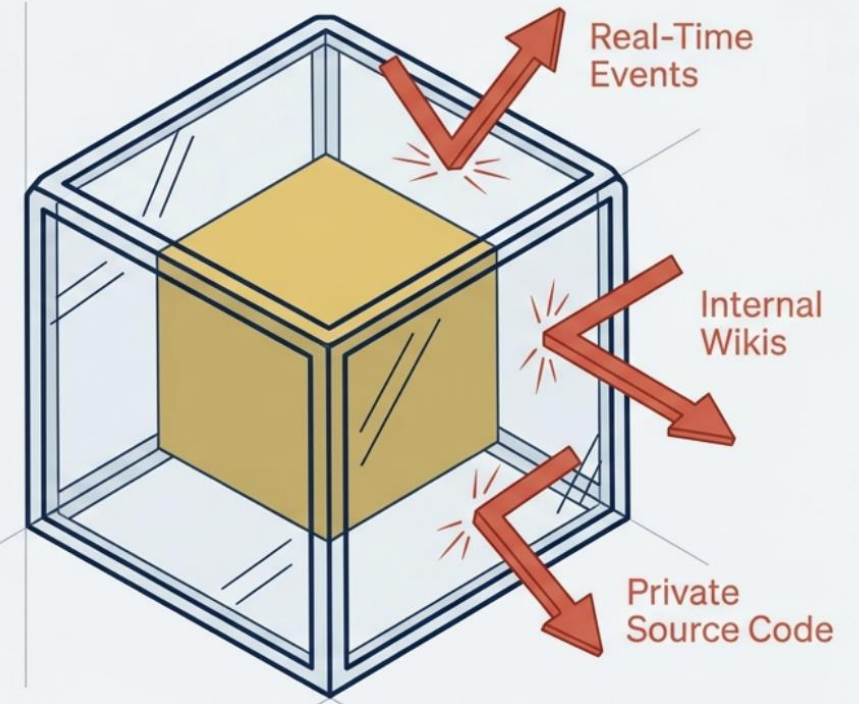
THE PROBLEM

LLMs have a **knowledge cutoff** — they can't read last week's FOMC minutes, a new BLS methodology note, or a working paper. Fine-tuning is expensive and still goes stale the moment the model ships.



THE SOLUTION: RAG

Retrieval-Augmented Generation gives the model access to a searchable document store. The model retrieves relevant passages at query time, then reasons over them. No fine-tuning. No re-training. Add new documents and the agent uses them immediately.



RAG VS LONG CONTEXT

Long Context

Retrieval-Augmented Generation

Data Scale

Bounded datasets

Infinite datasets

Responding Scope

Complex Global reasoning

Targeted information retrieval

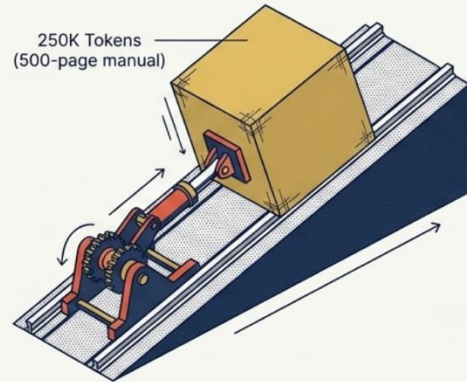
Infrastructure & Compute

Simplified Stack

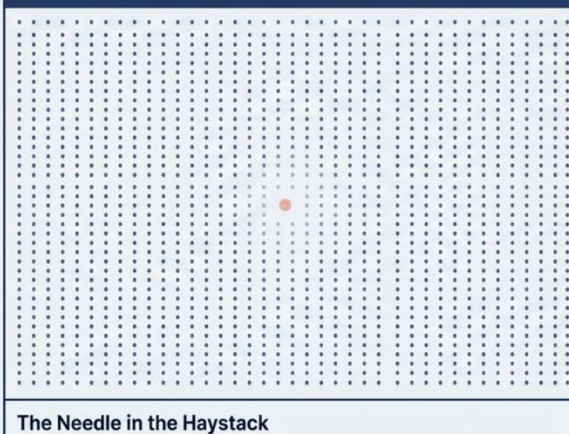
Complex stack

RAG VS LONG CONTEXT

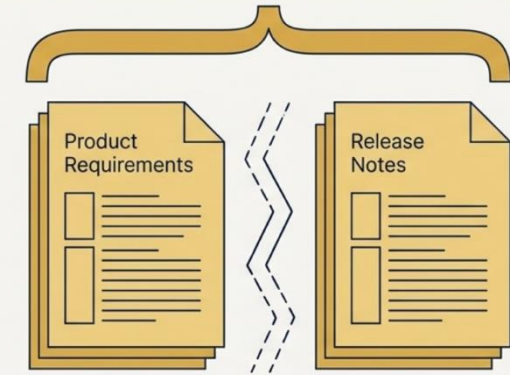
Long Context: The Rereading Tax



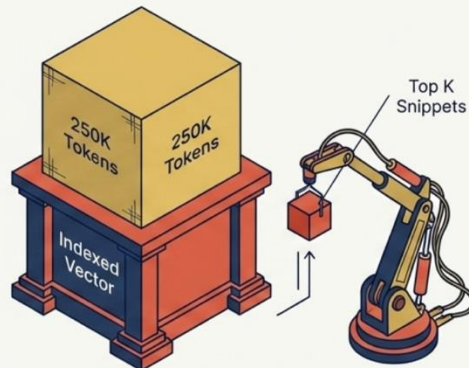
Long Context: Dilution Risk



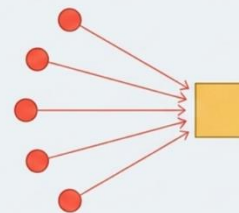
Long Context: The Whole Book Problem



RAG: Heavy Lifting Paid Once



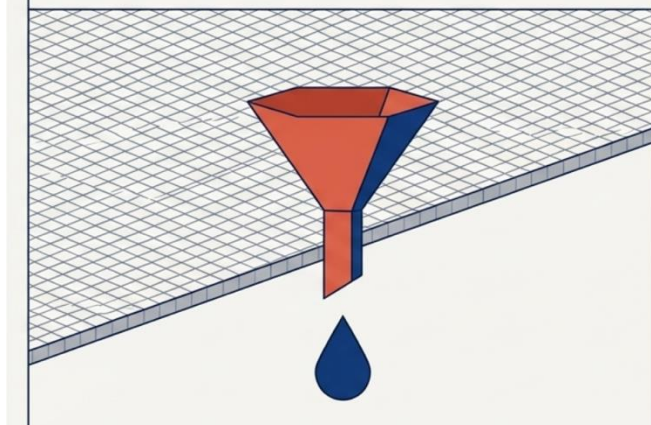
RAG: The Retrieval Lottery



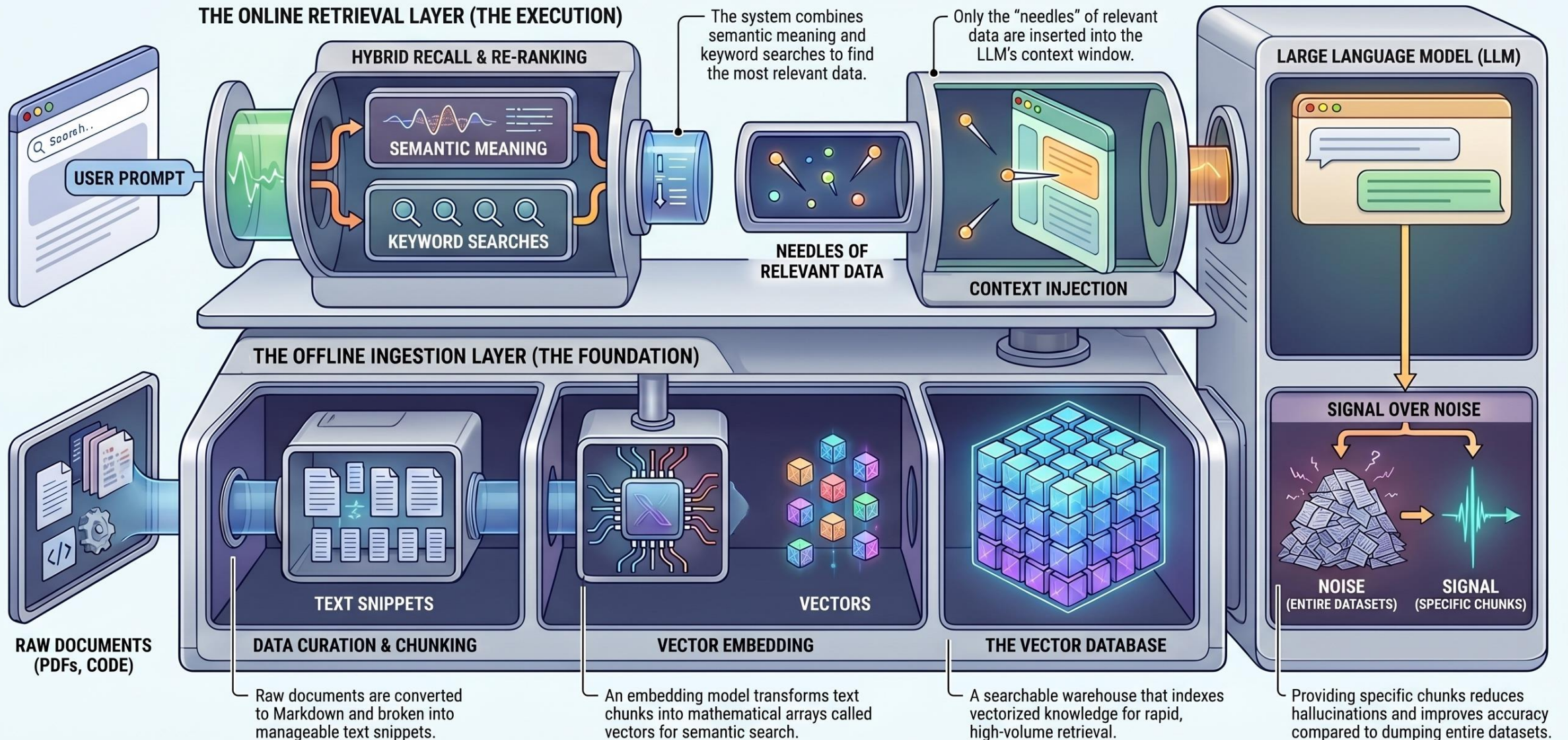
Silent Failure Risk

May 8, 2020

RAG: The Infinite Enterprise Dataset



INSIDE THE RAG PIPELINE

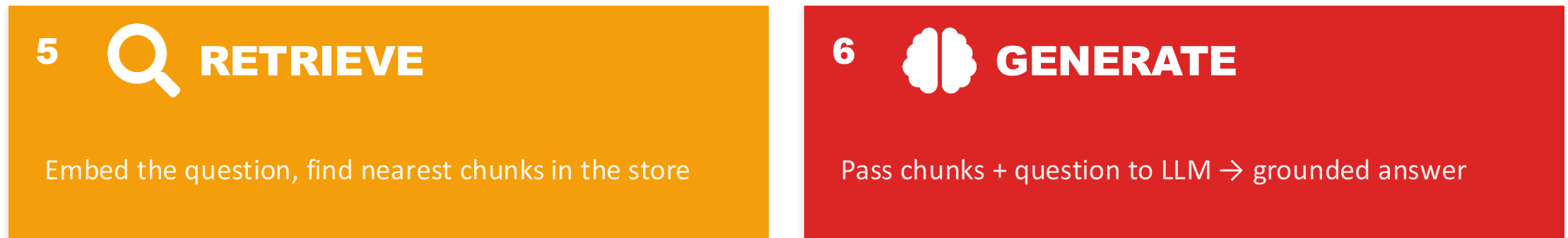


OVERVIEW OF RAG

Indexing Phase



Query Phase



KEY DESIGN DECISIONS

CHUNK



Chunk Size

- **Too small** Fragments lose context
- **Too large** Dilutes relevance
- **Sweet spot** 300-500 words for dense text

CHUNK



Chunk Overlap

- **Zero** Key sentences split across chunks
- **Too much** Wastes storage, duplicates hits
- **Sweet spot** 10-20% of chunk size

EMBED



Embedding Model

- **Small (local)** Fast, free, lower quality
- **Large (API)** Better semantics, costs \$
- **Sweet spot** Voyage / OpenAI for production

RETRIEVE



Results Count (k)

- **Too few ($k < 3$)** Misses relevant context
- **Too many ($k > 10$)** Floods LLM with noise
- **Sweet spot** 3-5 for focused questions

CHUNKING

§ 2 DOCUMENT LOADING



Why FOMC Minutes?

- Primary-source monetary policy signal
- Single phrases can move markets
- Rich, text-dense, analytically meaningful
- Publicly available
- Clear paragraph structure
- Real research use case



What We're Loading

JAN 2025

January meeting

MAR 2025

March meeting

MAY 2025

May meeting

~50,000 chars per meeting

Fetches live from [federalreserve.gov](https://www.federalreserve.gov)
HTML → cleaned plaintext with regex

Four common strategies — each optimized for a different document type

	Fixed Word Window Split every N words with overlap	BEST FOR Quick prototypes, uniform text
	Paragraph-Based Split on double newlines, merge small	BEST FOR Reports, minutes, articles
	Recursive Character Paragraphs → sentences → words	BEST FOR Mixed-format documents
	Semantic Chunking Use embeddings to find topic boundaries	BEST FOR Research papers, long reports

OURS

Paragraph-based with size constraints and overlap

```
chunk_by_paragraphs()
```

1. Split text on double newlines
→ list of paragraphs
2. Accumulate paragraphs into
a chunk buffer until
target_words is reached
3. If a single paragraph is
longer than max_words,
split it by sentences
4. Add overlap: prepend last
N words of previous chunk
to the start of the next

Our Parameters

target_words = 300

Ideal chunk size

max_words = 500

Hard upper limit

overlap_words = 50

Context bridge

Typical Output

Chunks per meeting: ~15-25

Median chunk size: ~280 words

Total chunks (3 mtgs): ~60

Avg embed dim: 384

VECTOR STORES

§ 3 E M B E D D I N G S

Specialized stores that index billions of vectors and return nearest neighbors in milliseconds

Database	Type	Strengths	Best Use Case
 ChromaDB	 Local / Embedded	Simple API, runs in-process, free	<i>Prototypes, workshops, small apps</i>
 Pinecone	Managed SaaS	Serverless, enterprise-grade, billions of vectors	<i>Production, high scale</i>
 Weaviate	Self-hosted or Cloud	Multi-modal, GraphQL, hybrid search	<i>Complex schemas, image + text</i>
 pgvector	Postgres extension	Use existing Postgres infra, ACID	<i>Teams already running Postgres</i>
 Qdrant	Open-source, Rust	Fast, great filtering, on-prem	<i>Latency-sensitive workloads</i>



We use ChromaDB for its in-process simplicity. The concepts transfer directly to any production vector store.



An **embedding** is a dense vector (e.g., 384 or 1024 floating-point numbers) that represents the **meaning** of a text passage. Passages with similar meaning have vectors that sit close together in the embedding space.

Example — three passages, three vectors

"The Fed raised rates by 25 basis points"

→ [0.12, -0.34, 0.56, ...]

A

"Interest rates were increased by a quarter point"

→ [0.11, -0.33, 0.55, ...]

A

"Housing starts fell 3% in March"

→ [-0.45, 0.22, -0.18, ...]

B

Vectors A_1 and A_2 are close (cosine similarity ≈ 0.95) because they mean the same thing. Vector B is far away — different topic entirely.

The Intuition

$$\cos(\theta) = (A \cdot B) / (|A| \cdot |B|)$$

- Measures the angle between two vectors, not their magnitudes
- Range: -1 (opposite) to +1 (identical)
- 0 means unrelated (perpendicular vectors)
- Works for any vector dimension (384, 1024, 3072...)
- Not affected by passage length — comparing a sentence to a paragraph is fine

Similarity Scale

1.00 Identical

Same exact text

0.85+ Very similar

Paraphrases, synonyms

0.60-0.85 Related

Same topic, different detail

0.30-0.60 Loosely related

Shares some concepts

<0.30 Unrelated

Different topics

SEMANTIC vs. KEYWORD SEARCH

QUERY: "labor market conditions"

KEYWORD SEARCH

Matches only exact terms:

- ...labor market conditions remained...
- ...the labor market tightened...
word order differs
- ...employment gains were robust...
no "labor" word
- ...hiring slowed in October...
no keyword match

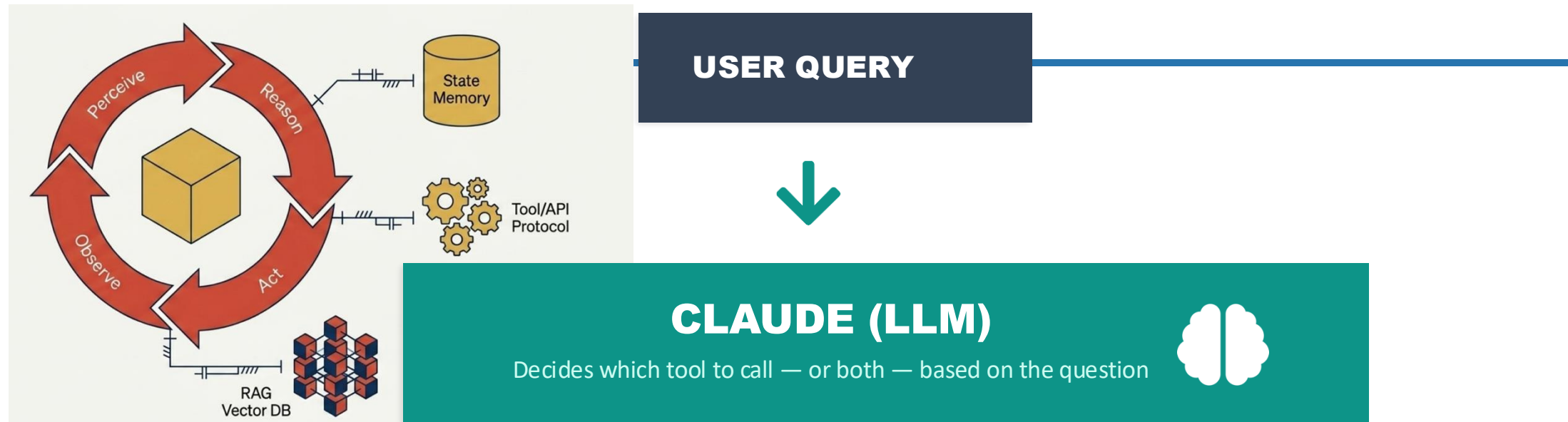
SEMANTIC SEARCH

Matches by meaning:

- ...labor market conditions remained... 0.92
- ...the labor market tightened... 0.87
- ...employment gains were robust... 0.81
- ...hiring slowed in October... 0.74

THE RAG AGENT

§ 3 BUILDING

**TOOL 1****get_fred_series**

Live economic data from FRED API

GDP · UNRATE · CPIAUCSL · DFF · T10Y2Y

From Session 1

TOOL 2**search_fomc_minutes**

Semantic search over FOMC minutes

Vector store → top-k relevant passages

NEW this session

ROLE

"You are a senior Federal Reserve policy analyst. Your job is to answer questions using two sources: FRED data and FOMC minutes."

**Always search FOMC minutes**

When a question involves Fed policy or committee assessments. Do not rely on training data for what the Fed said.

**Always pull FRED data**

When a question involves specific economic indicators. Do not quote numbers from memory — fetch the actual data.

**Cite everything**

Reference which meeting passages come from. Include dates, numbers, and units. Distinguish data from commentary.

**Admit gaps honestly**

If retrieved passages don't contain relevant information, say so. Never fabricate citations.

**TYPE 1**

"What did the FOMC say about inflation in their most recent meeting?"

→ `search_fomc_minutes` only

**TYPE 2**

"What has happened to the unemployment rate over the past 12 months?"

→ `get_fred_series` only

**TYPE 3**

"Based on the FOMC's labor market concerns, pull PAYEMS and UNRATE to evaluate them."

→ **BOTH tools, multiple calls**

The agent chooses its own strategy — and it can combine sources no single tool could deliver alone.

CROSS-MEETING ANALYSIS

How has the FOMC's language about inflation evolved across meetings? Identify shifts in tone or emphasis.

JAN 2025

Hawkish lean

"...inflation remains elevated..."



MAR 2025

Neutral / patient

"...progress toward 2% continues..."



MAY 2025

Dovish tilt

"...downside risks to employment..."



AGENT SYNTHESIS: Detects the shift from inflation focus → balanced concern → employment focus — all in a single tool-calling session.

RETRIEVAL QUALITY

§ 5 EVALUATION

A RAG system can fail at retrieval, at generation, or both — test each independently

LAYER 1**RETRIEVAL QUALITY**

Checks the R in RAG

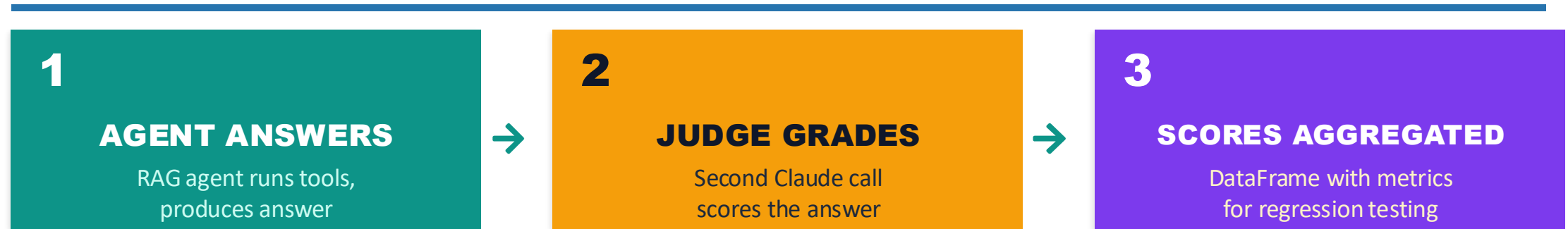
- Write test queries with known expected topics
- Check if the top-k retrieved chunks are actually relevant
- Measure mean cosine similarity at rank 1
- Flag queries with similarity < 0.5 for review
- Compare across chunking strategies
- Fast and cheap — no LLM calls required

LAYER 2**ANSWER QUALITY**

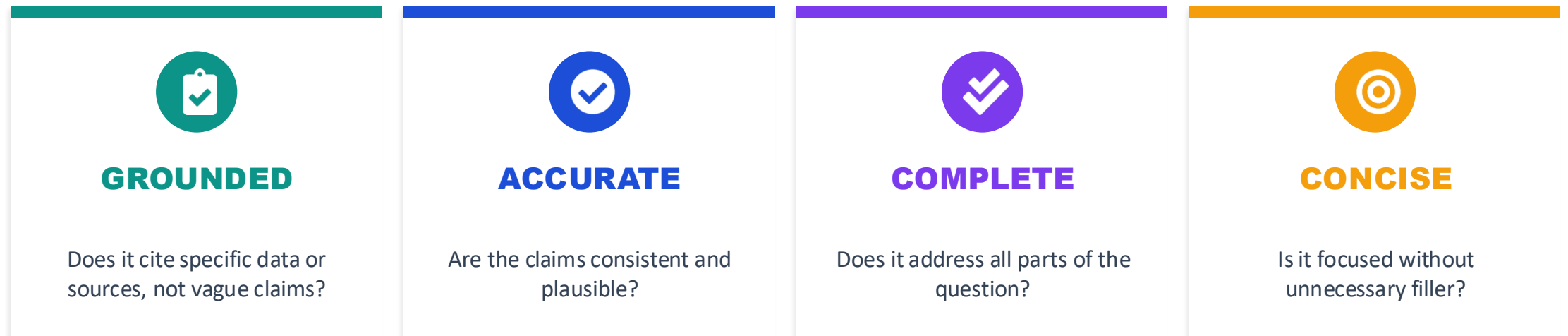
The LLM-as-judge pattern

- Ask the agent questions you can verify
- Use a second Claude call to grade the answers
- Score on 4 dimensions: grounded, accurate, complete, concise
- Include judge reasoning for debugging
- Aggregate across a test suite for regression checks
- Scales better than human evaluation

When human labeling is expensive, use a second model as a judge



Four scoring dimensions (1-5 scale)



EXERCISES

1

Tune Chunk Parameters

Build small (150w) and large (500w) collections, compare retrieval quality on the same 5 queries

10 min · Starter

2

Add Metadata Filtering

Enable meeting-specific search + add a `list_available_meetings` discovery tool

10 min · Starter

3

Multi-Document Comparison

Write a system prompt that forces the agent to search each meeting separately and identify hawkish/dovish shifts

15 min · Intermediate

4

Automated Eval Pipeline

Build `run_eval_suite()` that runs agent → grader → DataFrame for a test set of questions

15 min · Challenge

RAG = indexing + query

Load, chunk, embed, store (once). Retrieve + generate (every query).

Chunking is strategic

Size and overlap shape what the LLM sees. Paragraph-based preserves structure.

Embeddings encode meaning

Passages with similar meaning have vectors close in the embedding space.

Vector DBs do the heavy lifting

Cosine similarity search at scale — milliseconds across millions of chunks.

System prompts enforce grounding

Rules like "never cite from memory" turn a chatbot into an analyst.

Evaluate retrieval AND generation

A RAG system can fail at either layer. Test both, independently.